大模型推理
KV Cache 详解

原理 · 数学 · 工程实现 · Agent 应用

——《注意力模型与主流大模型详解》《大模型训练与推理工程详解》姊妹篇

写给工程师的推理系统内存管理读本

2026 年 10 月

目 录

第 1 章 问题的由来：不缓存，每一步都在重算历史 3

第 2 章 数学原理：为什么缓存是精确的，不是近似 6

第 3 章 显存与性能账本：KV cache 是推理的第一公民 11

第 4 章 工程实现（上）：分页、分配与回收 15

第 5 章 工程实现（下）：分布式与服务化 20

第 6 章 Agent 应用：上下文膨胀时代的 KV 工程 24

第 1 章 问题的由来：不缓存，每一步都在重算历史

1.1 自回归生成：一次只能走一步

大模型生成文本的方式是**自回归**的：给定已有的 t 个 token，算出第 $t+1$ 个 token，把它接到末尾，再算第 $t+2$ 个……如此循环，直到产出结束符。第二册已经讲过，这个"看历史"的动作由注意力机制完成：每个新 token 都会拿出自己的查询向量 q，去和**历史上每一个 token** 的键向量 k_i 做匹配，再对值向量 v_i 加权求和。

问题在于"历史"的处理方式。如果完全不做任何缓存，生成第 $t+1$ 个 token 时，前 t 个 token 的 k_i, v_i 向量会被**重新计算一遍**——尽管它们在此前的每一步里都已经算过了，而且数值一模一样。本章先把这种浪费算清楚，下一章再证明缓存为什么是对的。

1.2 玩具例子：生成第 4 个 token 要重算什么

沿用全书统一的玩具算例：一句话只有三个词——"猫""吃""鱼"，嵌入向量分别是 $x_1=(1,0)$、$x_2=(0,1)$、$x_3=(1,1)$；为口算方便，设三个投影矩阵都是单位阵 $W_Q=W_K=W_V=I$，于是每个词的查询、键、值向量都等于它的嵌入向量本身，维度 $d=2$，缩放因子 $\sqrt{d}=\sqrt{2}$。

现在模型要接着"猫 吃 鱼"生成第 4 个 token。注意力需要前 3 个 token 的键和值：

$$k_1=(1,0),\quad k_2=(0,1),\quad k_3=(1,1)$$

$$v_1=(1,0),\quad v_2=(0,1),\quad v_3=(1,1)$$

新 token（先叫它"鲜"）产生自己的查询 q_4，然后与 k_1,k_2,k_3 逐个打分、softmax、对 v 加权求和——这些第二册都算过。

关键在这里：k_1,k_2,k_3 **和** v_1,v_2,v_3 **这 6 个向量，在生成第 3 个 token 时算过一遍，生成第 4 个时又要算一遍；等生成第 5 个 token，它们还要再算第三遍**。词"猫"的 $k_1=(1,0)$ 从第 2 步开始，之后每一步都要被重新投影一次，直到生成结束。词出现得越早，被重复计算的次数就越多。

1.3 定量账：$T(T+1)/2$ 对 T

把这笔账一般化。假设生成 T 个 token（不含 prompt），统计 K/V 向量的投影计算总次数：

- 生成第 1 个新 token 时，需要 T_0 个历史 K/V（设 prompt 长度 $T_0=0$ 以简化，即从零开始生成，第 1 步需要 1 个）；

- 生成第 2 个时，需要 2 个；
- ……
- 生成第 T 个时，需要 T 个。

不缓存时，每步都把所需的 K/V 从头算一遍，总次数是等差数列求和：

$$1 + 2 + \cdots + T = \frac{T(T+1)}{2}$$

缓存时，每个 token 的 K/V 只在它第一次出现（或被生成）时算一次，之后直接读，总次数恰好是 T 次。两者的比值：

$$\frac{T(T+1)/2}{T} = \frac{T+1}{2}$$

代入 $T = 1000$：不缓存要约 50 万次投影，缓存只要 1000 次——**省了约 500 倍**，而且生成长度越长，倍数越大（$\approx T/2$）。注意这还只是 K/V 投影这一项的账；注意力的打分（q 与每个 k 做点积）无论是否缓存都是每步 $O(t)$ 次，缓存省不掉——第 2 章会仔细区分这一点。

生成 T 个 token	不缓存：K/V 投影次数	缓存后	节省倍数
$T = 10$	55	10	5.5 倍
$T = 100$	5050	100	50.5 倍
$T = 1000$	500500	1000	500.5 倍

1.4 直觉类比：笔记只做一次

一个贴切的类比：你在写一篇长文，每写一个新句子之前都要参考前面所有句子的"要点笔记"。

- **不缓存**：每写一个新句子，就把整篇文章从头到尾重读一遍，为每个旧句子**重新**做一份要点笔记，做完笔记才动笔写新句子。文章越长，每次重读越慢。
- **KV cache**：每个句子写完时顺手把它的要点笔记存档；写新句子时只翻开笔记本查阅，新句子的笔记写完后追加到笔记本末尾。重读全文的动作只发生一次（这就是第三册讲的 Prefill 阶段）。

1.5 KV cache 到底是什么

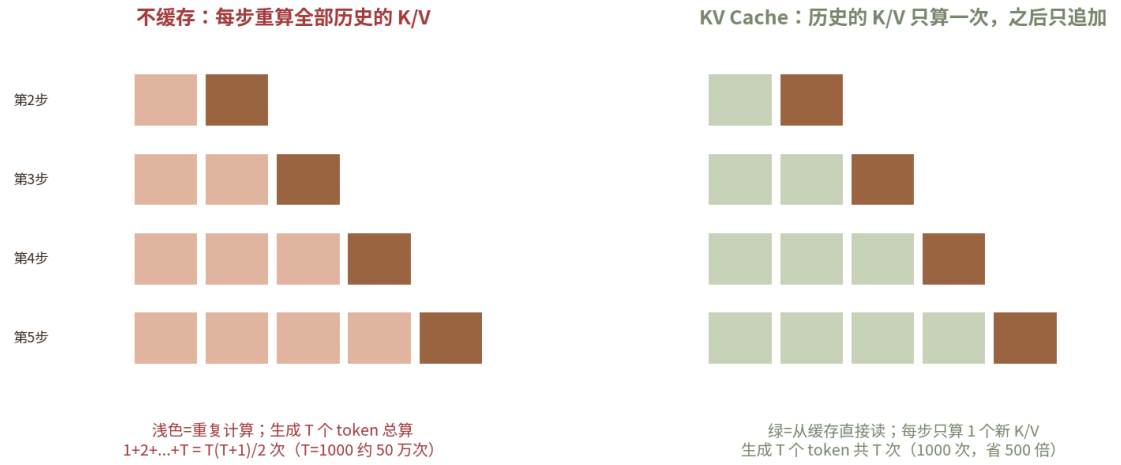
形式化地说，KV cache 是这样一个数据结构：对模型的**每一层**，按 token 顺序保存所有历史 token 的键向量 k_i 和值向量 v_i。第 t 步生成时，它包含 $t-1$ 条记录；每生成一个新 token，就向每一层追加一条新记录。

把它落到内存布局上看得更具体。一条记录是什么？以 7B 玩具口径（8 个 KV 头、head_dim $=128$、bf16）：每层的每条记录是 8 个 128 维向量（K）加 8 个 128 维向量（V），共 $2 \times 8 \times$

$128 = 2048$ 个 bf16 数，4 KB。32 层就是 32 份这样的记录，合 128 KB——这还只是一个 token 的"笔记"。推理引擎在初始化时就要决定：这 32 层 × 每 token 4 KB 的数据，按什么结构存放、怎么索引、满了怎么办。第 3、4 章的所有工程问题都从这里长出来。

三件事需要说清楚：

1. **它缓存的是中间产物，不是最终结果**。缓存里没有任何 token 的"输出表示"，只有 K 和 V 这两种"供别人查阅的资料"。新 token 自己的前向计算（Q/K/V 投影、注意力、FFN）一步都不能省。
2. **它逐层独立**。32 层的模型就有 32 份 K/V 序列要存——这正是第 3 章显存账里"乘以层数"的来源。
3. **它是显存换时间**。缓存不是没有代价的：它把计算压力换成了显存压力。第 3 章会看到，对长上下文服务来说，KV cache 的显存占用可以超过模型权重本身，成为推理系统的第一瓶颈。本书余下的篇幅，本质上就是在回答一个问题：**这份"笔记"怎么管才划算**。



1.6 常见的三个误解

在进入数学证明之前，先拆掉三个初学者常见的误解，它们在后文会反复用到：

误解一："KV cache 缓存了注意力的输出。" 不是。注意力输出 o_t 每步都不同（因为新查询 q_t 不同），没有任何东西可以缓存它。缓存的只是 k_i, v_i 这些"被查阅的资料"，查询永远现算。

误解二："缓存会让模型记不住长上下文。" 恰恰相反，缓存**就是**模型对长上下文的全部记忆。关掉缓存模型照样"记得"（每次都全量重算），只是慢；开着缓存丝毫不损失记忆能力。真正限制上下文长度的是位置编码的训练范围（第二册）和显存容量（第 3 章），不是缓存机制本身。

误解三："KV cache 是一种近似优化。" 不是，它是精确等价的。这一点值得用一整章来证明——也就是下一章。

第 2 章 数学原理：为什么缓存是精确的，不是近似

第 1 章说"缓存能省 500 倍"，细心的工程师会立刻警觉：省这么多，代价是什么？答案令人愉快：**什么代价都没有**。KV cache 不是近似、不是压缩、不是启发式——它和全量重算的结果逐位相同。本章把这件事证明给你看，然后顺手解决三个相关问题：缓存到底省掉了哪部分计算（很多人答错）、位置编码会不会破坏缓存、以及 GQA/MLA 这些"压缩版 cache"的数学。

2.1 单步注意力回顾

第二册推导过注意力的四步：打分、缩放、softmax、加权求和。写成一行：

$$o_t = \mathrm{softmax}\left(\frac{q_t K^\top}{\sqrt{d}}\right) V$$

其中 K 是把历史所有 token 的键向量按行拼成的矩阵（第 i 行是 k_i），V 同理；q_t 是当前 token 的查询。输出 o_t 是当前 token 的注意力结果。

2.2 关键观察：历史 K/V 一旦算过就永不再变

把注意力矩阵乘开。第 i 个 token 的键和值是：

$$k_i = x_i W_K, \qquad v_i = x_i W_V$$

这里的 x_i 在第 i 层是第 i 个位置的输入表示。**因果掩码**（第二册 3.2 节）保证了位置 i 的表示只由 $x_1, \ldots, x_i$ 决定——模型在计算位置 i 时根本"看不见" i 之后的任何 token。于是 k_i 和 v_i 也只由前 i 个 token 决定。

用玩具算例验证。"鱼"是第 3 个 token，$x_3 = (1, 1)$，$W_K = I$，所以 $k_3 = (1, 1)$。现在假设模型生成了第 4 个 token"鲜"，序列变成"猫 吃 鱼 鲜"。重新跑一遍前向：位置 3 的输入仍然只由"猫、吃、鱼"决定（因果掩码挡住了"鲜"），所以 k_3 还是 $(1, 1)$，一个数字都不差。

这就是缓存的合法性来源：**追加新 token 不会改变任何历史 token 的 K/V**。第 t 步算过的 k_i, v_i，到第 $t+1$、$t+100$ 步依然是同一个向量。

2.3 正确性的归纳法叙述

把上面的观察写成归纳法（高中数学里的数学归纳法结构）：

- **基础步**：第 1 步，缓存为空，算 k_1, v_1 并存入。缓存内容 = 前 1 个 token 的 K/V，正确。

- **归纳步**：假设第 t 步结束时缓存里恰好是前 t 个 token 的 K/V（且数值正确）。第 $t+1$ 步：从缓存读出 $k_1..k_t, v_1..v_t$，只需新算 k_{t+1}, v_{t+1} 并追加。由 2.2 的观察，读出的历史 K/V 与全量重算完全相同；新算的 k_{t+1}, v_{t+1} 本来就要算。于是注意力公式里的 K, V 与全量重算逐位相同，输出 o_{t+1} 也逐位相同。

因此：**KV cache 对模型输出零影响**。打开或关闭它，同一个模型在同一个 prompt 下生成的每一个 token 都一样（浮点累加顺序可能引起最后几位小数的差异，可忽略）。这与量化、蒸馏等"拿精度换速度"的技术有本质区别——缓存是纯赚。

用玩具数字把第 4 步完整走一遍

让"缓存 vs 全量重算"等价这件事落在数字上。第二册算过第 3 个 token"鱼"的注意力：它的查询是 $q_3 = (1, 1)$，与 $k_1 = (1, 0)$、$k_2 = (0, 1)$、$k_3 = (1, 1)$ 的点积分别是 1、1、2，除以 $\sqrt{2}$ 后 softmax 得权重 $0.248 / 0.248 / 0.504$，输出 $(0.752, 0.752)$。

现在假设模型生成了第 4 个 token，其嵌入恰好是 $x_4 = (2, 1)$（为演示而设的整数），于是 $q_4 = (2, 1)$。第 4 步的注意力打分：

$$s_{4,1} = \frac{q_4 \cdot k_1}{\sqrt{2}} = \frac{2 \times 1 + 1 \times 0}{\sqrt{2}} = \frac{2}{\sqrt{2}} = \sqrt{2} \approx 1.414$$

$$s_{4,2} = \frac{q_4 \cdot k_2}{\sqrt{2}} = \frac{2 \times 0 + 1 \times 1}{\sqrt{2}} = \frac{1}{\sqrt{2}} \approx 0.707$$

$$s_{4,3} = \frac{q_4 \cdot k_3}{\sqrt{2}} = \frac{2 \times 1 + 1 \times 1}{\sqrt{2}} = \frac{3}{\sqrt{2}} \approx 2.121$$

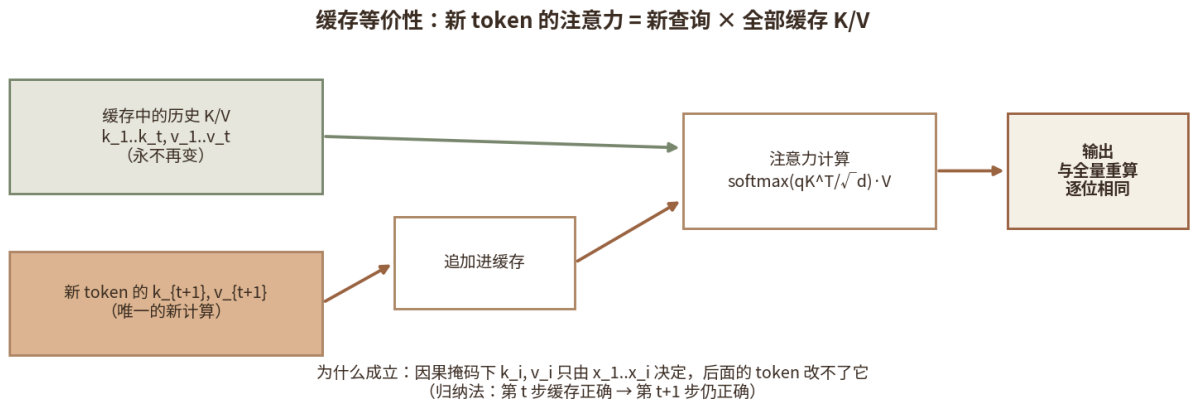
注意这里用到了 k_1, k_2, k_3——**它们全部读自缓存，和第 3 步算过的那三个向量是同一批内存里的同一批数**。softmax 三个分数（$e^{1.414} \approx 4.11$，$e^{0.707} \approx 2.03$，$e^{2.121} \approx 8.34$，和 ≈ 14.48）：

$$\alpha_1 \approx 0.284, \quad \alpha_2 \approx 0.140, \quad \alpha_3 \approx 0.576$$

加权求和（v_1, v_2, v_3 同样来自缓存）：

$$o_4 = 0.284 \times (1, 0) + 0.140 \times (0, 1) + 0.576 \times (1, 1) = (0.860, 0.716)$$

整个第 4 步只**新算**了三样东西：q_4、k_4、v_4（各一次向量乘矩阵），其余全部来自缓存。而"全量重算"版本除了这三样，还要把 k_1, k_2, k_3 和 v_1, v_2, v_3 重新投影 6 次——结果当然是同一个 $(0.860, 0.716)$，只是白花了 6 次矩阵乘。上下文越长，白花的次数按 1.3 节的平方曲线膨胀。



2.4 复杂度账：缓存省的是投影，不是打分

很多材料含糊地说"缓存把复杂度从 $O(T^2)$ 降到 $O(T)$"，这句话只对一半。把一次 Decode 步骤的计算拆成两部分，才能看清缓存到底省了什么：

第一部分：Q/K/V 投影。 每个 token 要算 $q = xW_Q$、$k = xW_K$、$v = xW_V$，各是一次向量乘矩阵，代价 $O(d^2)$（d 是隐藏维度）。

- 不缓存：第 t 步要为全部 t 个历史 token 重算投影，代价 $O(t \cdot d^2)$；全程合计 $O(T^2 \cdot d^2)$。
- 缓存：每步只为 1 个新 token 算投影，代价 $O(d^2)$；全程 $O(T \cdot d^2)$。**这部分从平方级降到线性级，是缓存的主要收益。**

第二部分：注意力打分与加权。 q_t 要和缓存里全部 t 个 k_i 做点积（$O(t \cdot d)$），softmax 后对 t 个 v_i 加权（$O(t \cdot d)$）。

- 无论是否缓存，这一步每步都是 $O(t \cdot d)$，全程 $O(T^2 \cdot d)$。**缓存对此无能为力**——新 token 必须"看"全部历史，这是注意力机制本身的定义。

工程上的结论：生成早期 t 小，投影占大头，缓存收益巨大；上下文极长时打分项 $O(t \cdot d)$ 开始显现，这正是长上下文推理仍然慢的原因之一（第 3 章会从访存角度再看一次）。

计算项	不缓存（每步）	缓存后（每步）	全程对比
K/V 投影	$O(t \cdot d^2)$	$O(d^2)$	$T^2 \to T$
注意力打分+加权	$O(t \cdot d)$	$O(t \cdot d)$	不变，$O(T^2 \cdot d)$
显存占用	$O(1)$（不算激活）	$O(t \cdot d)$ 每层	缓存的代价

2.5 位置编码的坑：RoPE 把位置烧进了 K

第二册讲过 RoPE（旋转位置编码）：位置 i 的键向量会被旋转 $\theta \cdot i$ 弧度。这意味着缓存里的 k_i 不是"干净的"内容向量，而是**内容和绝对位置的混合体**——旋转角度 $\theta \cdot i$ 已经固化在数值里了。

两个直接推论：

1. **缓存依然精确**。因为 token 的绝对位置 i 从生成就固定了，$\theta \cdot i$ 不变，旋转后的 k_i 也不变。2.2 的观察不受影响。
2. **但"截断重排"是危险的**。如果你把上下文中间一段删掉，后面所有 token 的绝对位置其实都前移了，严格说它们的 k_i 应该用新位置重新旋转——但缓存里存的仍是旧位置旋转的结果。轻度的截断（模型实践上往往能容忍）和"免费"之间没有等号。第 6 章讨论上下文压缩时会回到这一点。

2.6 压缩版 KV cache：GQA、MQA 与 MLA 的数学

既然缓存的代价是显存，能不能直接让 K/V 本身变小？三种主流方案，本质都是减少每层要存的向量数量或维度。

GQA（分组查询注意力）：让多个 Q 头共享同一份 K/V。7B 玩具口径：32 个 Q 头分成 8 组，每组 4 个 Q 头共用 1 个 KV 头，于是每层只存 8 份 K/V 而不是 32 份。数学上注意力公式不变，只是 q 的第 j 个头去匹配第 $\lfloor j/4 \rfloor$ 组的 k。显存直接除以 4。

MQA（多查询注意力）：GQA 的极端——所有 Q 头共享 1 份 K/V。显存除以 32，但所有头"看到的历史笔记完全相同"，表达能力下降最明显。

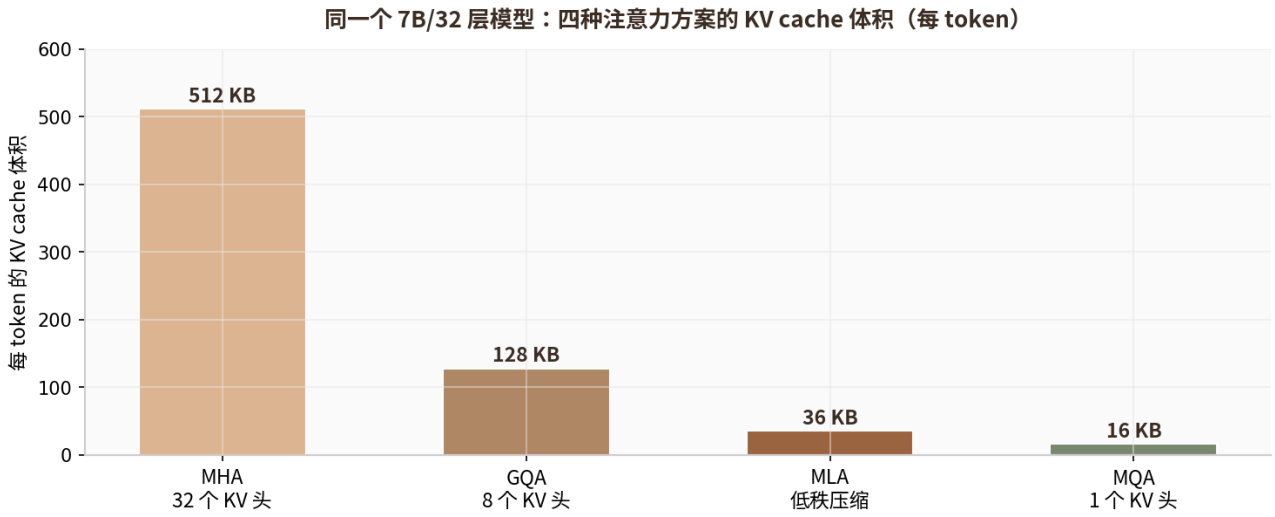
MLA（多头潜在注意力，DeepSeek 系列采用）：思路是**低秩压缩**。不直接存 k_i, v_i，而是存一个短得多的"潜在向量" $c_i = x_i W_{DKV}$（下投影矩阵把 d 维压到 576 维）；用到 K/V 时再展开：$k_i = c_i W_{UK}$，$v_i = c_i W_{UV}$。高中数学的直觉：一个 d 维大向量如果"本质上"躺在低维子空间里，就可以用短向量加两个固定小矩阵来表示——就像三维空间里一张平纸上的点，两个坐标就够描述。缓存只存 c_i（576 维），显存按维度比缩减。

统一算例口径（32 层、head_dim $= 128$、bf16）下四种方案的每 token 体积：

方案	每层每 token 字节数	全模型每 token	相对 MHA
MHA（32 KV 头）	$2 \times 32 \times 128 \times 2 = 16$ KB	512 KB	$1\times$
GQA（8 KV 头）	$2 \times 8 \times 128 \times 2 = 4$ KB	128 KB	$1/4$
MLA（576 维 latent）	$576 \times 2 = 1152$ B	约 36 KB	约 $1/14$
MQA（1 KV 头）	$2 \times 1 \times 128 \times 2 = 512$ B	16 KB	$1/32$

（表中"2"指 K 和 V 两份；MLA 的 latent 一份向量同时供 K 和 V 展开，故不乘 2。）

工程含义：KV cache 体积是模型架构设计的一阶约束。GQA 已成为 7B 级别的标配，MQA 常见于小模型，MLA 让超长上下文服务的成本降了一个量级。选型时省下的每一字节，都会乘以"层数 × 上下文长度 × 并发数"三个放大器——这是第 3 章的主题。



第 3 章 显存与性能账本：KV cache 是推理的第一公民

原理层的结论（第 2 章）是"缓存纯赚计算"，但工程上没有免费的午餐——省下的计算变成了显存里的常住人口。本章把 KV cache 的显存账和性能账算到底，你会看到：对在线推理服务来说，**KV cache 不是附属品，而是和模型权重平起平坐、甚至更大的显存消费者**，它的管理方式直接决定了服务的并发能力和成本。

3.1 显存公式：从一层到一个模型

逐层推导 7B 模型的 KV cache 体积（全书统一口径：32 层、GQA 8 个 KV 头、head_dim $= 128$、bf16 每元素 2 字节）。

每个历史 token，在每层要存两组向量：K 和 V，各是 8 头 × 128 维：

$$2 \times 8 \times 128 = 2048 \text{ 个元素}$$

$$2048 \times 2\ \text{B} = 4096\ \text{B} = 4\ \text{KB} \ / \ \text{token} \ / \ \text{层}$$

乘以 32 层：

$$4\ \text{KB} \times 32 = 128\ \text{KB} \ / \ \text{token}$$

一条 4096 token 的序列：

$$128\ \text{KB} \times 4096 = 512\ \text{MB}$$

一般化公式（背下来，容量规划天天用）：

$$\text{KV 字节数} = 2 \times L \times H_{kv} \times d_h \times B \times T$$

其中 L = 层数，H_{kv} = KV 头数，d_h = 头维度，B = 每元素字节数（bf16 为 2），T = 序列长度。注意两个结构特点：**对层数和上下文长度都是线性的**——这两个变量翻倍，显存就翻倍，没有任何免费午餐。

3.2 容量规划：一张 80 GB 卡能伺候多少并发

手算一张 80 GB 显存的卡跑 7B 推理的并发上限：

- 权重（bf16）：$7 \times 10^9 \times 2\ \text{B} = 14\ \text{GB}$
- 激活值、框架开销、通信缓冲等：预留约 6 GB
- 剩余给 KV cache：$80 - 14 - 6 = 60\ \text{GB}$

60 GB 能装多少 token 的 KV？

$$\frac{60 \times 1024\ \text{MB}}{128\ \text{KB}} = \frac{61440\ \text{MB}}{0.125\ \text{MB}} = 491520\ \text{个 token}$$

约 49 万 token。怎么分配给并发请求，取决于你按什么口径预留：

口径	每请求预留	并发上限	问题
按最大上下文 4096 预留	4096×128 KB $= 512$ MB	$491520/4096 = 120$	绝大多数请求用不满，浪费 80%+
按平均活跃 512 token	512×128 KB $= 64$ MB	$491520/512 = 960$	按需增长，利用率 8 倍于前者

120 对 960，**8 倍的并发差距**，模型、硬件、成本完全一样，差别只在显存管理方式。这就是第 4 章 PagedAttention 要解决的问题——先记住这个数字的量级。

3.3 性能账：Decode 每一步都在读 KV

第三册算过：Decode 阶段每生成一个 token，要把 14 GB 权重从 HBM 完整读一遍，理论下限约 $14/3000 \approx 4.7$ ms。现在把 KV cache 加进这笔账：注意力每一步还要读**当前序列的全部历史 KV**。

4k 上下文时：

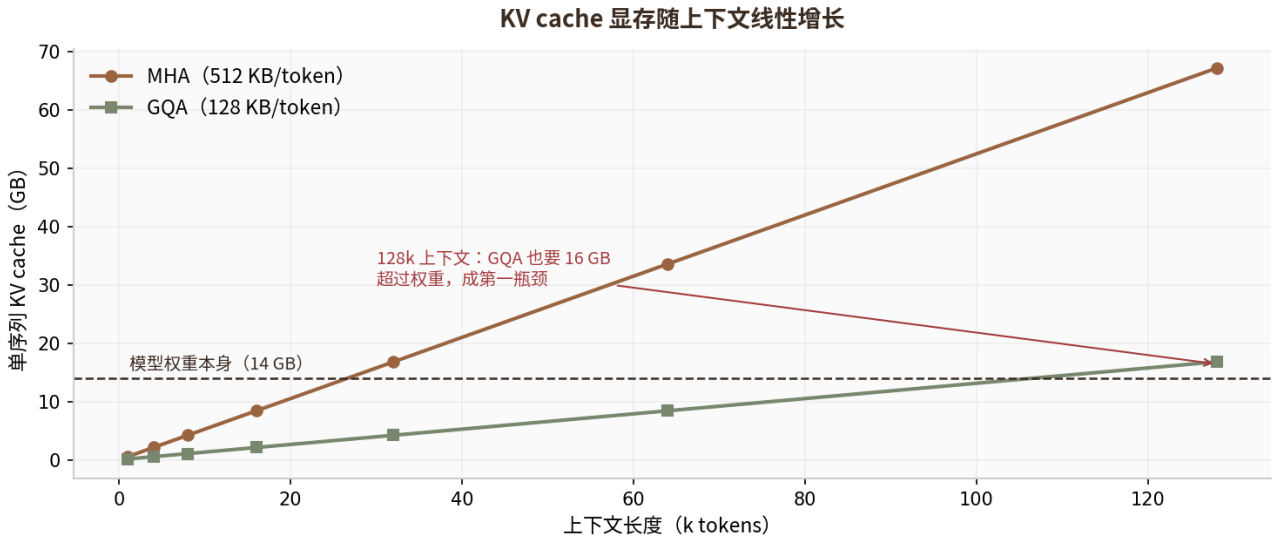
$$\frac{512\ \text{MB}}{3000\ \text{GB/s}} = \frac{0.512}{3000} \approx 0.17\ \text{ms}$$

只占 4.7 ms 的 3.6%，无关痛痒。但把上下文拉到 128k：

$$128\ \text{KB} \times 131072 = 16\ \text{GB}$$

$$\frac{16}{3000} \approx 5.3\ \text{ms}$$

KV 的读取时间（5.3 ms）已经超过了权重读取（4.7 ms）。此时每 token 延迟近乎翻倍，而且上下文再长还会继续线性恶化。这就是为什么"长上下文推理"是一个专门的工程难题：它把 Decode 从"读权重"主导变成了"读 KV"主导，优化重心完全不同（GQA/MLA、KV 量化、分层存储都是在打这个点）。



3.4 批量：权重读取被摊薄，KV 读取按户计费

把 batch 从 1 加到 B，每步的读取变成：

- **权重**：仍然只读一遍 14 GB，B 个序列共享——**摊薄**，batch 越大越划算；
- **KV cache**：B 个序列各自的 KV 都要读，合计 $B \times 128\ \text{KB} \times \bar{T}$（$\bar{T}$ 为平均上下文）——**按户计费**，无摊薄效果。

代入数字感受交叉点。设 batch $B = 64$、平均上下文 $\bar{T} = 2048$：

$$\text{KV 每步读取} = 64 \times 128\ \text{KB} \times 2048 = 16\ \text{GB}$$

恰好与权重读取（14 GB）相当——batch 64、2k 平均上下文就是这个 7B 模型的"权重与 KV 各半"平衡点。batch 再往上加，每步的总读取量翻倍增长而吞吐不再等比提升；batch 往下减，权重读取摊不薄，单 token 成本上升。

于是存在一个甜点区：batch 太小时权重读取主导，加大 batch 几乎免费提升吞吐；batch 大到 KV 读取总量逼近权重读取时，继续加 batch 收益递减，且显存先顶不住。第三册的连续批处理解决"时间维"的调度，本章的账本决定"空间维"的上限，两者在第 4、5 章汇合。

3.5 工程结论：像操作系统管内存一样管 KV

把三笔账合起来：

1. KV cache 显存对层数、上下文、并发全部线性，体量可达数十 GB；
2. 它的管理粒度（按最大长度预留 vs 按需分配）直接决定 8 倍量级的并发差距；
3. 它的读取带宽消耗在长上下文下会超过权重本身。

这套问题——分配、碎片、换出、共享、生命周期——和操作系统管理物理内存的问题**同构**。所以一个现代推理引擎（vLLM、SGLang 等）里，KV cache 管理子系统的地位，就相当于操作系统里的内存管理单元（MMU + 页表 + swap）。第 4 章开始，我们就按操作系统的思路把它拆开讲。

3.6 算账时最常见的三个坑

实务中 KV 显存估算出错，几乎都栽在这三处：

坑一：忘了乘 2（K 和 V 是两份）。 只算 K 忘了 V，账立刻小一半。口诀：公式 $2 \times L \times H_{kv} \times d_h \times B$ 开头的那个 2 就是"K 加 V"。

坑二：忘了乘层数。 KV cache 每层一份，32 层就是 32 份。"每 token 4 KB"和"每 token 128 KB"差的就是这个 32——前者是一层，后者才是全模型。

坑三：把 KV 头数当成注意力头数。 显存公式里是 H_{kv}（KV 头数），不是 Q 头数。GQA 模型的 32 个 Q 头配 8 个 KV 头，用 32 代公式会把显存高估 4 倍——这正是第 2.6 节 GQA 省显存的来源，算账时最容易混淆的地方。

第 4 章 工程实现（上）：分页、分配与回收

第 3 章末尾的结论：KV cache 管理 = 推理系统的内存管理子系统。本章把它按操作系统的思路拆开：先看朴素实现浪费在哪，再看 vLLM 的 PagedAttention 如何用"虚拟内存分页"的思路把浪费压到接近零，然后是分配器的完整工作流程、连续批处理下的调度与换出、前缀缓存、以及 KV 量化的工程面。

4.1 朴素实现的三大浪费

最直觉的实现：每个请求进来，按最大上下文长度一次性分配一块**连续**显存，请求结束整块释放。对照操作系统，这相当于"定长分区 + 连续分配"，三大浪费一目了然：

浪费	操作系统对应物	推理场景的表现
内部碎片	定长分区装不满	按 4096 预留，实际只用 512，87% 空置
外部碎片	连续分配找不到够大的空块	显存总空闲够，但没有连续大块，新请求进不来
时间浪费	批处理 barrier	一个 batch 里所有请求等最长的那个生成完，GPU 空转

第 3 章算过：预留口径下并发只有按需口径的 1/8。操作系统早在几十年前就用**分页 + 虚拟内存**解决了前两个问题——PagedAttention 就是把这套思想搬进 KV cache。

4.2 PagedAttention：KV cache 的虚拟内存

核心设计三步走：

1. **分块（block）**：KV cache 不再按"序列"分配，而是按固定大小的块分配，比如每块装 16 个 token 的 KV。按统一口径：$16 \times 128\ \text{KB} = 2$ MB/块。
2. **页表**：每个请求维护一张页表，记录"逻辑块 i → 物理块 j"的映射。逻辑上序列的 token 连续（第 0–15 个 token 在逻辑块 0），物理上各块可以散落在显存任意位置。
3. **按需增长**：Prefill 时按 prompt 实际长度申请 $\lceil T_{prompt}/16 \rceil$ 块；Decode 每生成满 16 个 token 才申请一块新块。

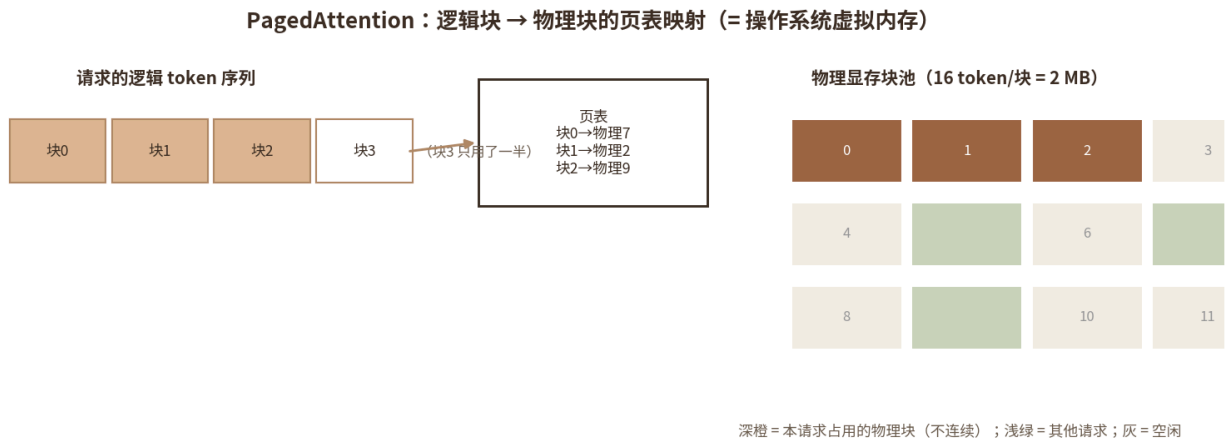
浪费账立刻改观：一个请求最多浪费**最后一个块装不满的部分**，平均半个块 $= 1$ MB——对比预留口径的数百 MB，碎片率从 80%+ 降到 1% 以内。外部碎片则彻底消失：所有块一样大，任何空闲块都能用。

用一个具体场景过一遍。某请求 prompt 长 2000 token，计划再生成 300 token。朴素实现按 4096 上限预留 $4096 \times 128\ \text{KB} = 512$ MB 连续显存。PagedAttention 的做法：Prefill 申请 $\lceil 2000/16 \rceil = 125$ 块（250 MB）；Decode 过程中每攒满 16 个新 token 申请一块，生成 300 token

再申请 $\lceil 300/16 \rceil = 19$ 块；最终占用 $144 \times 2\ \text{MB} = 288$ MB，且全部是实际使用（只有最后一块可能装不满）。**512 MB vs 288 MB**，单请求就省下 44%；乘以 3.2 节的并发账，就是集群级别的成本差距。

注意力计算时，kernel 按页表逐块取物理 KV 拼出逻辑序列——这正是 CPU 访问虚拟内存时 MMU 做的事，只不过翻译发生在 GPU kernel 里。

操作系统	PagedAttention
虚拟地址空间	请求的逻辑 token 序列
页（4 KB）	KV 块（16 token = 2 MB）
页表	逻辑块 → 物理块映射
物理页帧	显存中的物理块
按需调页	生成满 16 token 才申请新块
fork 的写时复制	beam search / 并行采样共享块



4.3 分配器的完整工作流程

一个请求从进来到离开，KV 块的流转如下：

1. **Prefill 前**：调度器计算需要的块数 $\lceil T_{prompt}/16 \rceil$，查空闲块池，够就分配、写入页表；不够则请求排队。
2. **Prefill**：一次性算完全部 prompt 的 K/V，按页表写入各物理块。
3. **Decode 每步**：新 token 的 K/V 追加到当前最后一块；块写满则申请新块、更新页表。
4. **共享（可选）**：beam search 或多候选采样时，分叉前的块被多个分支**共享**——页表允许多个逻辑块指向同一物理块，配引用计数；分支要修改共享块时先复制（写时复制，和 `fork()` 一样）。

5. **结束**：请求完成，引用计数归零的物理块全部归还空闲池。

这套机制用 Python 伪代码看核心逻辑（忽略 GPU 细节）：

```
class BlockPool:
    def __init__(self, num_blocks): self.free = list(range(num_blocks))
    def alloc(self): return self.free.pop() if self.free else None
    def release(self, blk): self.free.append(blk)

class Sequence:
    def __init__(self):
        self.page_table = []      # 逻辑块 -> 物理块
        self.tokens = 0
    def ensure_capacity(self, pool):
        need = (self.tokens + 15) // 16   # 需要的块数
        while len(self.page_table) < need:
            blk = pool.alloc()
            if blk is None: raise OutOfMemory  # 触发调度器 preempt
            self.page_table.append(blk)
```

注意 `ensure_capacity` 只在块写满时才真正申请——这就是"按需增长"的全部秘密；而 `alloc` 返回 `None` 的那一刻，就是把压力上传给调度器的信号（4.4 节的 preempt 由此触发）。

4.4 连续批处理下的调度与换出（preempt）

第三册讲过连续批处理：每个 iteration 调度器都可以让完成的请求退出、新请求插入。加上分页之后，调度器多了一个关键动作——**每轮先查显存水位**：

- 空闲块够 → 放行新请求；
- 不够 → 触发 **preempt**：牺牲某个低优先级请求，把它的 KV **换出**到 CPU 内存（类比 swap out），或干脆丢弃 KV 之后重算（recompute，因为 KV 可以由前缀重算出来——第 2 章的等价性保证了这安全）；
- 被换出的请求稍后有空位时再换回（swap in）继续生成。

"重算还是换出"的取舍取决于网络/PCIe 带宽与算力的相对价格：PCIe 约 64 GB/s，换出 512 MB 要约 8 ms；重算 4k token 的 Prefill 约 112 ms（28 ms/千 token）。**换出通常划算得多**，前提是 CPU 侧有地方放。

4.5 前缀缓存：system prompt 不必每人都算一遍

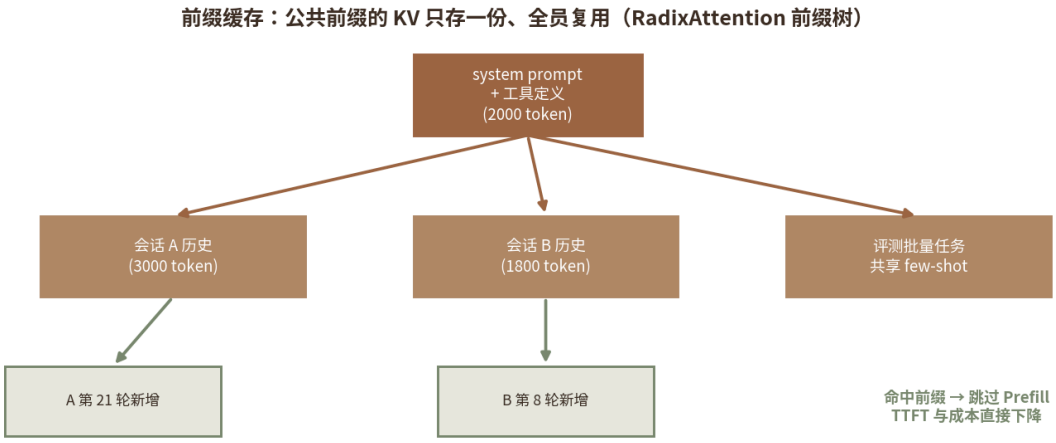
分页解决了"一个请求内"的浪费；下一个浪费在**请求之间**：同一个产品的所有请求共享同一段 system prompt，同一个 agent 的多轮调用共享全部历史，同一批评测任务共享 few-shot 模板。这些

公共前缀的 KV 内容完全相同（第 2 章：前缀相同则 K/V 相同），理应只算一次、只存一份。

RadixAttention（SGLang）的做法：把所有请求的 KV 组织成一棵**前缀树（radix tree）**，节点是 token 序列片段，边存对应的物理块。新请求进来时在树上做最长前缀匹配：

- 命中长度为 P 的前缀 → 直接复用对应块（引用计数 +1），Prefill 只算剩余的 $T - P$ 个增量 token；
- 未命中部分照常计算，算完把新路径挂到树上；
- 显存紧张时按 LRU 逐出叶节点。

用第 6 章的 agent 场景算账：公共前缀 2000 token，会话历史 10000 token，本轮新增 500 token。全量 Prefill 是 12000 token ≈ $12 \times 28 = 336$ ms；命中前缀后只算 500 token 增量 ≈ 14 ms。**TTFT 降一个量级**——前缀缓存是 agent 类负载最重要的单项优化。



4.6 KV cache 量化：显存再省一半（及以上）

第三册 7.3 讲过量化的数学（$x_{int} = \text{round}(x/s)$，$s = \max|x|/127$）。把它用到 KV cache 上：K 和 V 从 bf16（2 字节）压到 INT8（1 字节）甚至 INT4（0.5 字节），显存直接减半或再减半——第 3 章的 128 KB/token 变成 64 或 32 KB。

与权重量化相比，KV 量化有两点特殊：

1. **在线量化**：KV 是运行时动态产生的，scale 要边算边定（通常按块或按通道统计），比权重的离线量化多一步；
2. **K 比 V 更敏感**：经验上 K 的异常值（outlier）更多，直接 INT4 容易伤精度；常见配置是 K 用 INT8、V 用 INT4，或逐通道（per-channel）定 scale 抑制异常值。

工程含义一句话：KV 量化是"纯软件"的显存杠杆，不改模型、不改硬件，长上下文高并发场景下几乎必开；代价是少量精度损失和 kernel 复杂度。至此，单机的 KV 管理闭环完成；下一章把它放

进分布式集群。

4.7 一页小结：单机 KV 管理的四层机制

把本章的机制按"解决什么问题"叠起来，就是一张完整的单机架构图：

层	机制	解决的问题	操作系统类比
分配	分块 + 页表（PagedAttention）	内部/外部碎片	分页内存管理
调度	连续批处理 + 水位检查	时间维的 GPU 空转	时间片调度
换出	preempt / recompute	显存过载	swap / OOM 处理
复用	前缀树（RadixAttention）	请求间的重复计算	页面缓存（page cache）

四层全部就位，单机引擎才能把第 3 章的"60 GB 装 49 万 token"的理论容量真正兑现成线上并发。缺的任何一层，都对应一类确定性的浪费。

第 5 章 工程实现（下）：分布式与服务化

单机的 KV 管理闭环（分页、换出、前缀复用）在第 4 章完成。但真实服务是多卡、多机、多副本的：模型可能切在 8 张卡上，Prefill 和 Decode 可能分属两个集群，同一会话的多轮请求可能落到不同副本。本章回答分布式环境下 KV cache 的三个问题：**怎么切、怎么传、怎么找到它**。

5.1 张量并行下的 KV cache：天然按头切分

第三册讲过张量并行（TP）：把矩阵按列/行切到多卡。KV cache 的切分比权重更自然——注意力的 K/V 本来就是**按头组织**的，TP 沿头维度切，每卡只存自己分到的那几个头的 K/V。

统一算例：7B、GQA 8 个 KV 头、TP $= 8$：

$$\text{每卡每 token 每层} = 2 \times 1 \times 128 \times 2\ \text{B} = 512\ \text{B}$$

$$\text{每卡每 token} = 512\ \text{B} \times 32 = 16\ \text{KB}$$

8 卡合计仍是 128 KB/token——**完美分片，零冗余**。这也是为什么小 KV 头数的模型（GQA 8 头）配 TP=8 刚刚好：每卡恰好 1 头。反过来，MQA（1 个 KV 头）配 TP=8 就尴尬了：1 个头没法切 8 份，要么 8 卡各存一份完整副本（显存乘 8），要么加一层广播。选型时"KV 头数能否被 TP 度整除"是个真实约束。

5.2 PD 分离：KV 要过网络

第三册 7.6 讲了 PD 分离：Prefill（计算密集）和 Decode（访存密集）放不同集群。其代价就是 KV cache 必须从 Prefill 节点传到 Decode 节点。用统一数字算这笔传输账：

$$\frac{512\ \text{MB}}{50\ \text{GB/s}} = \frac{0.5}{50}\ \text{s} = 10\ \text{ms}$$

4k 上下文约 10 ms，相对 TTFT（数百 ms）可接受；但 128k 上下文是 16 GB，传输约 320 ms——已经超过 Prefill 计算本身。工程上的三个对策：

1. **按层流水线传输**：Prefill 逐层产出 KV，算完第 1 层就立刻发第 1 层，传输与计算重叠，尾延迟只剩最后一层的传输；
2. **压缩后传**：KV 量化（第 4 章）在这里有双重收益——传输量也减半；
3. **拓扑感知调度**：尽量把同一请求的 Prefill 和 Decode 放在同一 RDMA 域内，减少跨交换机跳数。

5.3 cache-aware 路由：把请求送到记得它的副本

多副本部署时，负载均衡器的一个新决策：**同一会话的后续请求，路由到持有其前缀缓存的那个副本**。命中则 Prefill 只做增量（第 4 章：336 ms → 14 ms 量级），不命中则全量重算。

这与分布式缓存的路由问题同构，对应物直白：

分布式缓存	KV cache 路由
一致性哈希定位 key	按会话 ID 哈希定位副本
缓存命中	前缀树命中，TTFT 大降
缓存穿透	副本漂移后全量 Prefill
热点 key	热门 system prompt 人人命中

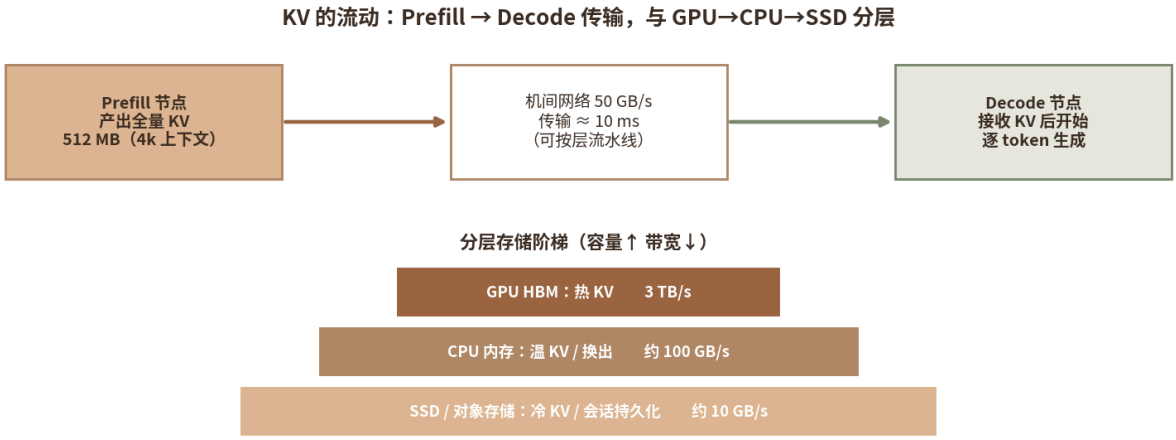
权衡也熟悉：严格会话黏性命中率最高，但负载可能不均；纯轮询负载最均，但命中率归零。实践是"软亲和"：优先路由到持有缓存的副本，过载时允许降级到其他副本（牺牲 TTFT 保成功率），后台副本间异步同步热点前缀。

5.4 分层存储：HBM → CPU → SSD 的阶梯

第 4 章的 preempt 换出打开了第一层：KV 不一定要在 GPU 上。完整的存储阶梯：

层级	带宽（量级）	容量（量级）	放什么
GPU HBM	3 TB/s	数十 GB	正在生成的活跃序列的 KV
CPU 内存	约 100 GB/s	数百 GB～TB	被换出的会话、温前缀
SSD / 对象存储	约 10 GB/s	近乎无限	冷会话快照、跨重启的会话持久化

调度策略与 CPU 缓存 hierarchy 同理：**热数据留上层，冷数据沉下层，靠预取隐藏上下搬运的延迟**。典型场景：用户隔了 5 分钟回来继续对话——会话 KV 早已沉到 CPU 甚至 SSD；系统在其打字时就把 KV 预取回 GPU，用户按回车时 TTFT 依然漂亮。对 agent 场景（第 6 章），这一层决定了"会话暂停一小时后恢复"的成本。



5.5 多租户与隔离

共享集群上，KV cache 是第一需要配额的资源：

- **会话级隔离**：每个会话的页表、前缀树命名空间独立，防止串话；
- **显存配额**：按租户/优先级划分块池，高优先级租户的 preempt 可以抢占低优先级；
- **降级阶梯**：流量突发时的有序降级——先缩 max_tokens 上限，再降低前缀缓存保留时长，最后触发 preempt；每一步都对应明确的 SLO 代价，事先写进预案而不是临场拍脑袋。

5.6 全景：一次多轮对话的 KV 生命周期

把第 4、5 章串起来，跟踪一个真实请求：

1. **创建**：第一轮请求到达，路由层按会话 ID 选副本；分配器为 2000 token 的 system prompt + 问题申请 126 个块；Prefill 写入全部 KV。
2. **追加**：Decode 每 16 token 申请一块，逐 token 生成回答；生成的 token 也进缓存。
3. **共享**：第二轮请求到来，前缀树命中"system prompt + 第一轮问答"共 3500 token，只 Prefill 新增的 200 token 问题；前轮块引用计数 +1。
4. **换出**：集群水位告急，该会话 10 分钟无活动，KV 被异步搬到 CPU 内存，块归还池。
5. **回收/复活**：用户一小时后回来，KV 已从 SSD 快照预取回 GPU；若快照也过期，则全量重算 Prefill——对系统来说，"重建"只是 Prefill 的另一种触发方式。

KV cache 的一生，就是一个有完整的分配、共享、换出、持久化、回收协议的内存对象的一生。

5.7 一页小结：分布式 KV 的三个关键词

- **切（5.1）**：TP 按头切分，零冗余；约束是 KV 头数要能被 TP 度整除。
- **传（5.2）**：PD 分离把 KV 搬上网络，按层流水线 + 量化是把传输藏起来的两件武器。
- **找（5.3）**：cache-aware 路由决定多副本下谁能吃到前缀复用的红利。

再加上分层存储（5.4）兜底容量、多租户配额（5.5）兜底公平，集群级 KV 管理就闭环了。第 6 章看它在当下最热的一类负载——agent——里面临的新压力。

第 6 章 Agent 应用：上下文膨胀时代的 KV 工程

前五章建立的认知：KV cache 是精确缓存（第 2 章）、是显存第一公民（第 3 章）、有成熟的分页与分层管理（第 4、5 章）。本章回答一个更现实的问题：**当负载从"一问一答的聊天"变成"几十轮循环的 agent"时，这些机制顶不顶得住？怎么用对它们？** Agent 应用的负载特征与普通聊天截然不同，而 KV 复用水平几乎决定其成本与延迟。

6.1 Agent 负载的特征：上下文线性膨胀，前缀高度重叠

一个典型的 ReAct 式 agent 循环：每一轮产生"观察（工具返回）→ 思考 → 动作（工具调用）"三段新内容，追加到上下文末尾，再调用一次模型。用统一算例（A5）：

- system prompt + 工具定义：2000 token（恒定）；
- 每轮新增约 500 token；
- 第 n 轮时上下文长度 $\approx 2000 + 500(n-1)$ token。

第 20 轮：$2000 + 500 \times 19 = 11500$ token，KV cache 约 $11500 \times 128\ \text{KB} \approx 1.4$ GB/会话。账本身不可怕——可怕的是调用模式：**相邻两轮之间，上下文的 95% 以上逐字相同**（第 20 轮与第 19 轮只差最后 500 token）。如果每轮都全量 Prefill，第 20 轮的 TTFT 是 $11.5 \times 28 \approx 322$ ms，且 20 轮累计 Prefill 总量约 $2000 \times 20 + 500 \times (0 + 1 + \cdots + 19) = 87500$ token，其中约九成是重复计算。

结论：**agent 场景下，前缀缓存命中率就是成本的第一杠杆**。没有前缀缓存，agent 的推理成本比同 token 量的聊天高出近一个数量级。

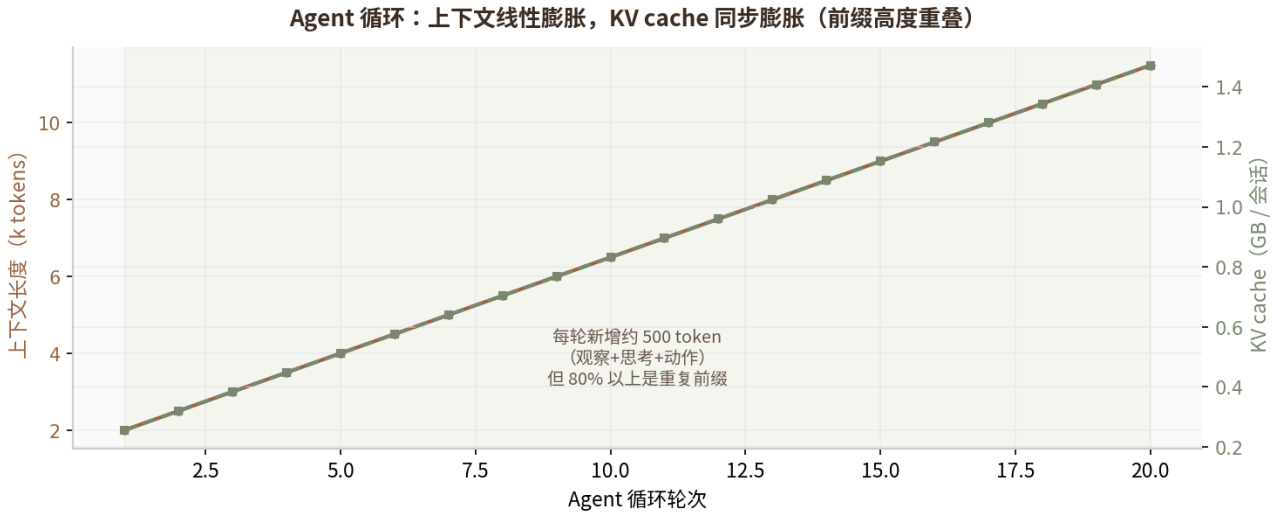
把累计账算完整。20 轮循环、全量 Prefill 的累计计算量：

$$\sum_{n=1}^{20} \left(2000 + 500(n-1)\right) = 20 \times 2000 + 500 \times 190 = 135000 \text{ token}$$

命中前缀（每轮只算增量）时：

$$2000 + 20 \times 500 = 12000 \text{ token}$$

$135000 / 12000 \approx 11$ 倍——**前缀缓存把一个 20 轮 agent 任务的 Prefill 成本降到原来的 1/11**。这就是为什么所有主流推理服务都把前缀缓存默认为 agent 负载打开。



6.2 会话状态管理：agent 框架与推理服务的配合

Agent 的一次任务横跨几十次模型调用，中间可能隔几十秒（等工具执行）。这段时间里 KV 放哪？三个层次的选择：

1. **GPU 驻留**：会话活跃期间一直占着 HBM。延迟最优，但 1.4 GB/会话 × 数千并发会话根本放不下——只适合短任务。
2. **分层换出**（第 5.4 节）：工具执行的空档期把 KV 换到 CPU，调用前预取回。主流方案，对框架透明。
3. **持久化快照**：跨小时/跨天的任务（如"明早继续"的异步 agent），KV 落 SSD；恢复时直接加载，省去从第一轮重放。

工程上还要处理**断线续传**：进程崩溃后，KV 快照 = 会话的检查点（checkpoint），恢复 = 加载快照 + 重放最后未完成的增量。这与第三册训练的 checkpoint 思想一脉相承，只是粒度从"全集群"缩到"单会话"。

6.3 前缀设计：一条能立刻省钱的纪律

前缀缓存的匹配规则是**从第一个 token 开始**逐字匹配（第 4.5 节的前缀树）。这推出一条简单但常被违反的纪律：

> **不变的内容放最前，变动的内容放最后。**

推荐顺序：system prompt → 工具定义 → few-shot 示例 → 会话历史 → 本轮新内容。反面教材：有些框架把当前时间戳、随机 request ID 拼在 prompt 开头——**第一个 token 就变了，整棵树全部失配**，前缀缓存形同虚设，TTFT 和成本同时爆炸。同理，工具定义列表如果每轮按字典序重排、或

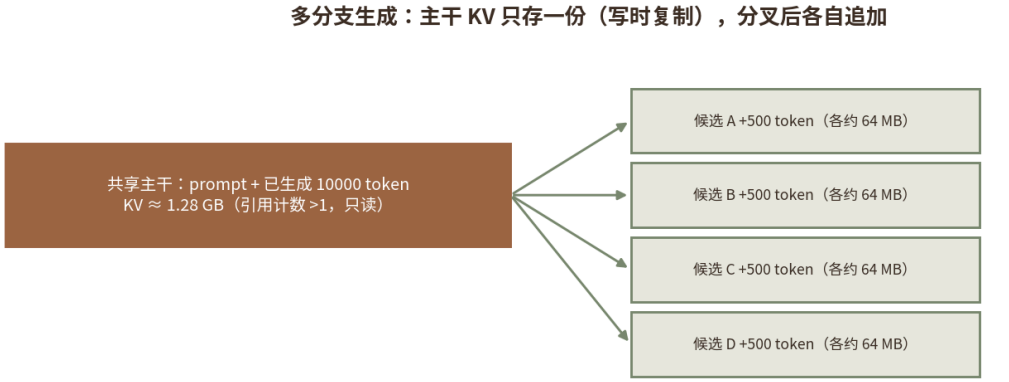
动态拼接描述文本，也会摧毁缓存。审计一个 agent 系统的 KV 效率，第一步就是把每轮发出的 prompt 做 diff：除末尾增量外不应有任何差异。

6.4 多分支探索：一份主干，N 份枝叶

Agent 的高级用法常涉及并行探索：tree-of-thought 同时展开多条推理路径、对同一状态采样多个候选动作再择优、beam search。这些分支**共享同一前缀**，KV 结构上是一棵树：

- 主干（prompt + 已确认历史）10000 token 的 KV 只存一份：≈ 1.28 GB，引用计数等于分支数；
- 每个分支从分叉点开始各自追加：4 个分支各 500 token $\approx 4 \times 64$ MB $= 256$ MB；
- 总计 ≈ 1.5 GB，对比 4 份完整副本的 $4 \times 1.3 \approx 5.4$ GB，省 72%。

实现上靠第 4.3 节的写时复制：分支读到共享块时直接读，要写（追加跨界）时复制最后一一块。分支被剪枝时引用计数递减，归零回收。对 agent 开发者，这意味着**并行采样几乎是免费的**——显存按"主干+枝叶"计费而不是按份计费，可以放心用"采样 4 个动作选最优"这类策略。



6.5 上下文压缩与截断：不是免费的午餐

上下文终究会撞上模型窗口。常见压缩手段及其 KV 后果：

手段	KV 后果	注意点
滑动窗口（丢弃最旧轮次）	被丢弃段的 KV 回收	窗口边界处位置不连续，RoPE 语义有缝（第 2.5 节）
摘要压缩（旧历史换成摘要）	几乎全部 KV 失效，重新 Prefill	摘要插在开头 = 新前缀，缓存从头重建
重要 token 保留（如 H2O）	KV 变成稀疏子集	需要 kernel 支持非连续 KV，收益与任务强相关

重点说第二条，因为它反直觉："把 10000 token 历史压成 500 token 摘要"听起来省算力，但对 KV cache 是一次**推倒重建**——新上下文的每个 token 位置都变了，旧 KV 一个都不能用，要付一次完整 Prefill。所以摘要策略适合"长任务的新阶段起点"，不适合每轮都做。滑动窗口的位置缝隙问题，工程上的妥协是让窗口边界对齐到块、并接受轻微的位置语义偏差——这也是长窗口模型训练时引入位置插值的原因之一。

6.6 落地清单与全书收束

把前五章的机制变成 agent 系统的可操作 checklist：

1. **前缀纪律**：不变内容前置，prompt 逐轮 diff 审计，消灭开头的时间戳/随机数；
2. **会话亲和**：路由层按会话 ID 软亲和，命中率纳入核心监控（目标：agent 负载 >90%）；
3. **轮次预算**：给 agent 循环设硬性轮次上限与 token 预算，触发即总结收尾——既防失控也防 KV 爆显存；
4. **分支共享**：并行探索走共享前缀的多分支 API，而不是发 N 个独立请求；
5. **压缩策略分级**：滑动窗口为常态，摘要压缩只在阶段边界做；
6. **监控三指标**：前缀命中率、KV 显存水位、TTFT 分解（前缀命中部分 vs 增量部分）——这三个数能定位 90% 的 agent 推理性能问题。

回看全书：第 1 章发现重复计算，第 2 章证明缓存精确合法，第 3 章算清它的显存与带宽账，第 4 章用操作系统的分页思想管好它，第 5 章把它扩展到集群与服务，第 6 章落到当下最吃这套机制的负载——agent。KV cache 这个"笔记本"，从一行数学等价性出发，最终长成了推理系统里最像操作系统的那个子系统。理解它，就理解了现代大模型推理工程的半壁江山。